Evolving Adaptive Format-Ensemble Policies for Cost-Efficient LLM Question Answering

Status and contribution

Completed frozen evaluation on 486 screened OpenBookQA official-test questions. Candidate course-project contribution: search-only complementary-error mutation for a bounded adaptive presentation policy. No publication-level novelty claim. Evolution, adaptive sampling, cascades, and format sensitivity have precedents in literature notes (../../../docs/literature.md).

Design

Qwen2.5-3B-Instruct with one fixed manual instruction. Five predefined formats, including two rotations with canonical answer remapping. Strict single-letter parsing; invalid responses contribute no vote, all-invalid returns an explicit failure. Ties choose the smallest canonical option index. Policies use only requested predictions, agreement, invalid indicators and remaining calls. Maximum five calls, depth five, fifteen query nodes. All source splits are disjoint; exact stem and option-set overlap with previous ARC material was removed. Previous ARC final outcomes are exploratory and are not used as fresh test evidence.

Random and guided evolution share seeds, initial populations, crossover schedule, representation, population and evaluated-candidate slots. NSGA-II-style non-dominated sorting and crowding select accuracy/call trade-offs. Guided mutation samples formats proportional to smoothed counts of search failures rescued, using failures reaching a node if at least twelve exist, otherwise all search failures. Greedy has the same search-only statistics and exhaustive one-step proposals. This is not a reproduction of a named prior optimizer. Fixed subsets are enumerated (31); fixed-three is validation-selected among ten triples; agreement follows the original five-format order and stops on two matching valid votes. Validation-only selection maximizes accuracy under each call cap, then minimizes cost, tree size and hash; an infeasible cap uses lowest cost. Achieved final costs may differ from caps.

Results

At cap 1, guided minus random accuracy was +0.0000 (conditional paired 95% interval +0.0000, +0.0000); achieved calls 1.000 versus 1.000.
At cap 2, guided minus random accuracy was +0.0000 (conditional paired 95% interval +0.0000, +0.0000); achieved calls 1.000 versus 1.000.
At cap 3, guided minus random accuracy was -0.0004 (conditional paired 95% interval -0.0025, +0.0016); achieved calls 2.585 versus 2.218.

Guidance did not improve mean accuracy over random mutation at the declared operating points. A negative result is retained.

Method	Seed	Cap	Accuracy	Achieved calls	Tokens	Invalid
single	0	1.0	0.8025	1.000	101.8	0.0041
single	0	2.0	0.8025	1.000	101.8	0.0041
single	0	3.0	0.8025	1.000	101.8	0.0041
fixed_three	0	1.0	0.8169	3.000	305.4	0.0000
fixed_three	0	2.0	0.8169	3.000	305.4	0.0000
fixed_three	0	3.0	0.8169	3.000	305.4	0.0000
fixed_full	0	1.0	0.7901	5.000	516.0	0.0000
fixed_full	0	2.0	0.7901	5.000	516.0	0.0000
fixed_full	0	3.0	0.7901	5.000	516.0	0.0000
fixed_subset	0	1.0	0.8025	1.000	101.8	0.0041
fixed_subset	0	2.0	0.8025	1.000	101.8	0.0041
fixed_subset	0	3.0	0.8169	3.000	305.4	0.0000
agreement	0	1.0	0.7757	2.084	215.4	0.0000
agreement	0	2.0	0.7757	2.084	215.4	0.0000
agreement	0	3.0	0.7757	2.084	215.4	0.0000
greedy	0	1.0	0.8025	1.000	101.8	0.0041
greedy	0	2.0	0.8025	1.000	101.8	0.0041
greedy	0	3.0	0.8169	2.253	229.1	0.0000
random	11	1.0	0.8025	1.000	101.8	0.0041
random	11	2.0	0.8025	1.000	101.8	0.0041
random	11	3.0	0.8169	2.185	222.0	0.0000
guided	11	1.0	0.8025	1.000	101.8	0.0041
guided	11	2.0	0.8025	1.000	101.8	0.0041
guided	11	3.0	0.8169	3.004	309.0	0.0000
random	29	1.0	0.8025	1.000	101.8	0.0041
random	29	2.0	0.8025	1.000	101.8	0.0041
random	29	3.0	0.8169	2.189	222.5	0.0000
guided	29	1.0	0.8025	1.000	101.8	0.0041
guided	29	2.0	0.8025	1.000	101.8	0.0041
guided	29	3.0	0.8169	3.000	305.4	0.0000

random	47	1.0	0.8025	1.000	101.8	0.0041
random	47	2.0	0.8025	1.000	101.8	0.0041
random	47	3.0	0.8169	2.185	222.1	0.0000
guided	47	1.0	0.8025	1.000	101.8	0.0041
guided	47	2.0	0.8025	1.000	101.8	0.0041
guided	47	3.0	0.8128	2.549	259.7	0.0021
random	71	1.0	0.8025	1.000	101.8	0.0041
random	71	2.0	0.8025	1.000	101.8	0.0041
random	71	3.0	0.8169	2.185	222.0	0.0000
guided	71	1.0	0.8025	1.000	101.8	0.0041
guided	71	2.0	0.8025	1.000	101.8	0.0041
guided	71	3.0	0.8169	2.185	222.0	0.0000
random	101	1.0	0.8025	1.000	101.8	0.0041
random	101	2.0	0.8025	1.000	101.8	0.0041
random	101	3.0	0.8148	2.346	242.4	0.0000
guided	101	1.0	0.8025	1.000	101.8	0.0041
guided	101	2.0	0.8025	1.000	101.8	0.0041
guided	101	3.0	0.8169	2.185	222.1	0.0000

Uncertainty and analysis

Paired bootstrap resamples whole test questions, averaging fitted-seed values within each question before guided/control contrasts. Intervals are conditional on the fitted policies. Search-seed variability is separately exported; seed-question pairs are not independent observations. Overlapping marginal intervals are not a significance test. Comparisons at common declared caps are approximate cost comparisons; achieved calls are reported explicitly. In addition, 11 of 15 guided operating points have same-seed random comparators within 0.15 validation calls, chosen before test using validation costs only. Their achieved test cost gaps and paired differences are in validation_cost_matched_pairs.csv; unmatched cases are flagged rather than called cost matched. The empirical test frontier is descriptive and never used to select or replace policies. Inspect paired_differences.csv and seed_variation.csv. No automatic claim of superiority or novelty is made.

Costs

Actual generation calls/tokens/inference seconds: {"development": {"completed_unique_calls": 320, "executed_here_calls": 0, "reused_preflight_calls": 320, "input_tokens": 31904, "output_tokens": 640, "inference_seconds": 44.37123119086027, "invalid_outputs": 0}, "search": {"completed_unique_calls": 2560, "executed_here_calls": 2560, "reused_preflight_calls": 0, "input_tokens": 257151, "output_tokens": 5121, "inference_seconds": 371.0443683862686, "invalid_outputs": 1}, "validation": {"completed_unique_calls": 2445, "executed_here_calls": 2445, "reused_preflight_calls": 0, "input_tokens": 250281, "output_tokens": 4890, "inference_seconds": 353.9433799088001, "invalid_outputs": 0}, "test": {"completed_unique_calls": 2430, "executed_here_calls": 2430, "reused_preflight_calls": 0, "input_tokens": 245905, "output_tokens": 4869, "inference_seconds": 352.8246075287461, "invalid_outputs": 3}}. Fields distinguish executed-here from imported development calls; imported responses are counted once across the project. Preparation physical calls/tokens, including archived batch-context diagnostics, are separately exported in preparation_costs.json: {"evidence_passes": {"v1_batch16": {"development_unique_generation": {"completed_unique_calls": 320, "executed_here_calls": 320, "reused_preflight_calls": 0, "input_tokens": 31904, "output_tokens": 640, "inference_seconds": 5.23032010346651, "invalid_outputs": 0}, "latest_live_verification_calls": 49, "latest_live_input_tokens": 5967, "latest_live_output_tokens": 98, "additional_initial_live_calls": 40, "additional_initial_live_input_tokens": 4887, "additional_initial_live_output_tokens": 80}, "v2_singleton": {"development_unique_generation": {"completed_unique_calls": 320, "executed_here_calls": 320, "reused_preflight_calls": 0, "input_tokens": 31904, "output_tokens": 640, "inference_seconds": 44.37123119086027, "invalid_outputs": 0}, "latest_live_verification_calls": 48, "latest_live_input_tokens": 5863, "latest_live_output_tokens": 96}}, "note": "Main imports the v2 development responses; do not add imported main rows a second time. V1 is archived diagnostic cost. Live verification and repeats are intentional additional physical calls."}. CPU search seconds: {"guided_101": 2.076836682856083, "random_101": 1.5700785666704178, "random_29": 1.456561490893364, "guided_11": 2.3839107528328896, "guided_47": 2.31987277418375, "guided_29": 2.3148265033960342, "random_11": 1.632506251335144, "guided_71": 3.0825169757008553, "random_47": 1.6068797335028648, "random_71": 1.6235048919916153, "greedy": 0.4680333063006401}. Simulated requested calls/tokens count the requests made by each frozen policy. Development live latency and repeat measurements are in runs/preflight/live_verification.json, separate from cached deployment simulation. Full matrix generation cannot establish adaptive latency savings. Attempts include failed/interrupted batch information. Completed-cache timings exclude model loading, CPU preparation and downtime; CPU search excludes GPU generation. Interrupted uncommitted batches can consume uncounted work, exposed in attempts.

Limitations

One model and filtered dataset; finite formats; no full human semantic audit; exact duplicate screening misses paraphrases and prior pretraining exposure. A finite search pool and validation selection among many policies may overfit; per-seed variability is descriptive. Development live measurements are small and do not establish production latency. Deterministic greedy decoding can still vary across batch contexts/hardware; replay is

conditional on recorded outputs. Residual repeat/live discrepancies are retained, not silently overwritten. No withheld-format generalization claim. The supplied DSAI6707 brief requires a PDF report, code, results, and at least twenty relevant papers from 2024–2026; the literature appendix and submission checklist map these requirements.

Reproducibility

FROZEN.json hashes selected policies, configuration, source/data/package fingerprint and analysis rules before test generation. responses.sqlite retains raw responses, identities, tokens, and durable attempts. Run/resume commands and setup are in README.md; only COMPLETE.json marks completion. Exclusions and hashes are in data/manifest.json and data/exclusions.json. Editable report accompanies this Markdown.

Frozen experimental protocol

This project evaluates closed-book question answering on screened OpenBookQA questions, without the annotated supporting facts. The source revision is `388097ea7776314e93a529163e0fea805b8a6454`. Source splits: 64 official-train development questions, a disjoint 512 official-train search questions, all 489 eligible official-validation questions, and all 486 eligible official-test questions. Test priority deduplication excludes normalized question-stem and option-set matches against all 1,437 previous screened ARC source questions. The previous project's pilot is exploratory; all previous final predictions are ineligible as confirmatory evidence. No final-test inference or performance inspection occurs before freeze.

The five formats are original vertical A–D, inline `(A)` alternatives, bullets, and cyclic option rotations by one and two. Every option's original index is its canonical answer identity. Displayed letters are mapped back using each rendering's permutation. Question text and option strings are preserved. Duplicate options, suspicious missing media, position references and duplicate source questions are excluded. These conservative heuristics can remove valid items and miss semantic near duplicates; no exhaustive human semantic audit is claimed. Development previews are in data/preview.md; test previews are written after freezing.

The instruction is fixed for every format, policy and baseline: “Solve the science multiple-choice question carefully. Choose the single best answer based on its content, regardless of layout or option order. ” A shared response contract requests exactly one uppercase displayed option letter. Qwen/Qwen2.5-3B-Instruct and its tokenizer are pinned to `aa8e72537993ba99e69dfaafa59ed015b17504d1`. Targets use greedy decoding, eight new tokens maximum, no input truncation, BF16, deterministic Torch settings, SDPA, left padding, singleton batches, seed zero and a 1,536 input-token ceiling. Strict parsing accepts A–D only after trimming outer whitespace. Invalid outputs are retained and receive no vote. Ties choose the lowest canonical option index. All-invalid returns -1/None, counts as incorrect, and never silently guesses.

A policy is a binary tree of query nodes with at most fifteen nodes, depth five and five calls. A node queries an unused format and branches on an observable predicate: any answer having two votes, all queried responses valid and agreeing (at least two), any invalid output, vote margin at least two, latest two answers differing, at most one unused format, or an always-false predicate used for fixed sequences. A null child stops with the predefined vote. Query paths cannot repeat formats. Both scalar live execution and vector replay use only observed answers/state, with ground truth scored outside inference. Labels are absent from stored cache request identities and inference callback results. Counterfactual unqueried responses are accessed only during search, never policy deployment.

Controls: validation-selected best single; validation-selected best triple among ten fixed triples; original-order full-five vote; original-order agreement stop as soon as two valid answers match; all 31 fixed subsets; and search-only greedy construction across fixed penalties 0, .01, .025, .05, .1, .2, .5. Fixed triples and full voting are reported even when over a deployment cap, so their achieved costs remain explicit. All fixed-subset candidates use ascending format index, which also fixes request order. Greedy starts from each possible first format and adds one-step query extensions, searching branch predicates, to maximize search accuracy minus a penalty times calls. Labels used for greedy remain search-only.

Random and guided evolutionary search use five seeds (11,29,47,71,101), population 32, forty generations and 1,312 evaluated slots per condition/seed, including the initial population. Duplicate policy proposals consume slots, preserving matched proposal budgets. Initial populations share five singles, fixed agreement and full voting plus seeded random trees. Every fourth offspring uses common subtree crossover before mutation. Parents use a size-two rank tournament with deterministic hash tie-breaking. Survival uses nondominated accuracy-maximization/call-minimization fronts and crowding distance. This is NSGA-II-style selection with a simple front-construction algorithm, not a faithful NSGA-II implementation or a claim of a novel multiobjective optimizer. Tree size is bounded, not a third objective.

Mutation chooses a node or stop site and uniformly chooses a format change, predicate change or structural change when applicable. Random mutation chooses formats uniformly. Guided mutation differs only in how the replacement/extension root format is sampled: probability proportional to one plus the number of search failures it answers correctly. Search failures reaching that site are used if at least twelve exist; otherwise

all failures in the search pool are used. If the current policy makes no errors, smoothing produces uniform weights. Structural descendant generation and all other operators remain identical. Repair removes repeated formats on a path, clips depth/node count and fixes invalid path formats deterministically. Guidance logs its search origin, reaching-example count and fallback. The fallback and smoothing are declared before fresh-data inference.

All generated candidates, including the common initial policies, may enter validation-only selection. This makes finite-validation overfitting a material limit; validation is a selection set, never an unbiased performance estimate. Validation cost caps are 1, 2 and 3 mean calls. Select the highest validation accuracy at or below a cap, then lower calls, smaller size and lexicographic policy hash. If the candidate set has no feasible policy, choose lowest cost then accuracy/size/hash and report infeasibility through achieved cost. Keep every declared method/seed/cap; do not choose the best seed on test. Freeze the complete policy exports, config, code/data/packages and analysis rules before generating any final-test response. Hashes include the selected trees, and resume refuses changed fingerprints.

Fresh OpenBookQA development feasibility uses 64 questions and five formats. A diagnostic stop occurs if oracle coverage minus best-format accuracy is below .03, agreement mean calls exceeds 4.5 or invalid response fraction exceeds .05. These are pragmatic feasibility gates, not significance tests. Oracle coverage is never a deployment baseline. The preliminary gate on 100 old ARC pilot questions is explicitly exploratory. A separate twelve-development-question real-model smoke uses four search, four selection and four replay/live verification items, with no official-test questions. It checks cache generation, actual random/guided mutation, validation selection and identical-setting replay versus live execution. Any discrepancy is saved with repeated identical singleton inputs; unresolved live/replay answer or query differences block launch. Identical-input repeat outcomes and small live timing samples are reported without claiming production latency benefits.

Main execution is incremental: development gate, fresh search/validation matrices, CPU methods, validation selection, hashed freeze, fresh final matrix, only frozen-policy evaluation, whole-question uncertainty, figures and report. A single process lock refuses duplicate workflows. SQLite commits completed batches; attempts can remain interrupted_unknown after a crash, so physical work may exceed unique completed calls. Search checkpoints contain population, archive, scores, random state, evaluated-slot count and generation. Completed records and RNG state survive resume. Missing responses are regenerated under identical keys; configuration/code/data/package drift requires a new run directory. Only COMPLETE.json confirms successful reports and completion; diagnostic or incomplete markers are not results. The launcher also holds a separate lock covering smoke/launch/main.

Final metrics are accuracy, mean requested calls, input-plus-output requested tokens, call-count distribution and all-invalid result rate. Per-response invalids are separately counted. Paired method predictions are retained by whole source question. The test frontier is descriptive only and cannot reselect deployment policies. Comparisons are guided minus random, greedy, selected subsets, agreement and single at each declared cap, with both achieved costs reported. Bootstrap 2,000 paired whole-question replicates with seed 9182; for seed-averaged comparisons, average fitted-seed values within a question before resampling. Report search-seed mean/SD/range separately. Conditional test intervals do not include retraining uncertainty, and overlapping marginal intervals are never interpreted as a significance test. No seed-question independence assumption or automatic positive conclusion is permitted.

Actual costs are kept separate: completed matrix-generation calls/tokens/inference time and durable attempt counts; CPU search seconds; simulated requested deployment calls/tokens; and development live wall-time measurements, raw outputs and repeats. Runtime estimates exclude setup, model loading, future contention, retries and interruption downtime unless explicitly included. Shared GPU placement is disclosed: >=20 GiB free, utilization <=60% at launch, one visible UUID, 40% Torch allocator cap, loader >=18 GiB free check. Existing jobs are preserved. Availability is observed, not reserved; memory/throughput can change.

A supplementary cost comparison is frozen using validation achieved cost only: for each guided seed/cap, choose the same-seed random export among the three declared caps with the closest validation mean calls, tie-breaking by smaller cap and export ID. Mark a match only when the validation call difference is at most 0.15. Report every pair's achieved final call difference and conditional paired differences; unmatched pairs are flagged. Test cost or accuracy never chooses a comparator. The main common-cap comparisons remain available even when validation matching fails.

Protocol v2 changes batch size from sixteen to one before any fresh final inference. The expanded v1 smoke found one original-format cached/live disagreement; two identical singleton repeats matched the live answer, and agreement stopping took two live versus three replayed calls. All v1 records remain in runs/preflight_batch16. Singleton generation matches the live request execution context; the v2 smoke and development gate are rerun with new cache identities. This is a development-only numerical-reproducibility correction, not final-performance tuning. Earlier batch-sixteen diagnostic accuracies are exploratory and kept separate. Main responses may still exhibit residual hardware/runtime nondeterminism; repeats are evidence, not proof of universal determinism.

Literature synthesis and verified primary-source references

Checked 28 October 2026 against primary author preprints, publisher records, and conference proceedings. Twenty-two relevant works below have a 2024–2026 preprint or publication; a 2023 preprint counts only when its 2024 conference publication is identified. Foundational older references are listed separately and do not count toward twenty. Verification covers bibliographic records and described mechanisms, not an independent replication of their numerical claims. No paper's headline gain is used as an expected outcome for this project.

Evolution and feedback: what should the operator search?

Evolutionary LLM research operates on several representations. EvoPrompt [1] and Promptbreeder [2] evolve instruction text; the latter also evolves mutation prompts. OPRO [10] uses scored histories to propose improvements, while EvoLLM [3] investigates in-context optimization operators themselves. EoH [4] evolves heuristic ideas and executable programs, Eureka [6] evolves reward code, and ADAS [7] searches agent programs. Model-merging evolution [8] changes model composition rather than query behavior. The survey [9] situates both EA-enhanced LLMs and LLM-enhanced EAs. Together these establish that an evolutionary wrapper alone is insufficient as a contribution: the useful question is what information the operator exploits and whether that improves search under controlled budgets.

GEPA [5] is a particularly relevant warning against broad novelty claims: it uses feedback from trajectories and combines complementary lessons across candidate lineages. TextGrad [13] likewise exploits richer feedback than a scalar score. DSPy [11] and MIPRO [12] optimize compound LM programs, exposing credit assignment and validation-selection questions. Their representations and feedback costs are much broader than this project's compact trees. Our narrow candidate contribution is a rescue-count distribution for choosing a presentation at a mutation site, compared with uniform random mutation using identical executable representation and evaluated-candidate budgets. It uses search labels only and does not ask an LLM to invent a prompt. It should be described as an ablation of a practical integration of existing ideas, not a new general optimizer or a GEPA reproduction.

Strength: targeted feedback could reduce unproductive format proposals. Limitation: rescue of a wrong terminal prediction does not guarantee that adding the response changes a majority vote, and correlated mistakes may persist. State-specific counts can be noisy, so the fallback is declared rather than tuned using validation outcomes. Greedy construction receives the same search statistics, testing whether evolution adds anything beyond local construction.

Presentation sensitivity, response contracts, and ensemble diversity

Sclar et al. [14] show that apparently superficial formatting choices can affect model performance and introduce FormatSpread. Wang et al. [15] distinguish generated text answers from first-token likelihood evaluation for multiple-choice robustness. These findings motivate a small fixed pool, exact preservation of option text, canonical identity remapping, and an explicit parser. This experiment evaluates generated letters; it does not infer that a first-token evaluation would behave identically.

MoA [18] aggregates model outputs in layers, whereas Self-MoA [19] cautions that adding different models can sacrifice source quality even while increasing diversity. Soft self-consistency [20] considers score-based selection when exact answer matches are sparse. Their common lesson is that raw diversity is not equivalent to useful accuracy gain. The exploratory pilot's oracle coverage indicates only the existence of correct answers in the pool; it is not implementable without labels. Fixed-subset enumeration and single-format selection are therefore essential controls. Our four-option canonical answers make plurality voting natural, without extra LLM aggregation calls or probability features. No claim is made that format diversity is equivalent to model diversity.

Strength: equivalent presentations isolate a reproducible source of disagreement within one frozen model. Limits: a finite pool can miss other useful perturbations, rotations may alter behavior despite preserving intended semantics, and all simple ensembles can fail together. Oracle coverage is only an upper bound and cannot be a deployment comparator.

Adaptive computation and cost: why stopping is already established

RouteLLM [16] learns model routing from preference data. RouterBench [17] makes cached inference outcomes useful for systematic routing evaluation. Snell et al. [21] emphasize that effective test-time computation depends on problem difficulty. ReASC [22] extends count-based adaptive sampling using response reliability. The older Adaptive-Consistency and FrugalGPT papers (below) directly establish agreement stopping and learned cascades before this project. Consequently neither adaptive allocation nor cheap early exit is novel here.

This project restricts observations to queried canonical predictions, vote counts, invalid indicators, queried formats and remaining calls. It uses no learned confidence estimator, hidden-state features, reference answer at inference, or access to unqueried predictions. This makes policy behavior auditable but can limit headroom relative to richer routers. Equal call caps do not imply equal tokens or latency: formatting changes input length, batching changes execution, and generated responses can vary with execution context. We separately report matrix-generation work, CPU search, requested deployment tokens/calls, and a small measured live sample. A cached matrix can support reproducible policy comparisons but cannot establish a production latency improvement.

Research gap and falsifiable assessment

A modest, testable opportunity is whether search-only error complementarity helps an evolutionary operator discover compact presentation policies that retain accuracy with fewer requested calls. The controlled comparison is guided versus uniform mutation over five search seeds at predeclared validation cost caps. The stronger practical question is whether either beats greedy and validation-selected fixed subsets at the achieved final costs. Failure on either comparison is retained. Existing work makes publication-level novelty uncertain; this review motivates a course-project contribution and explicitly does not establish a first-in-literature claim.

Recent references: 22 works

ID	Reference and verified year/status	Primary source	Role and distinction
1	Guo et al. EvoPrompt: Connecting LLMs with Evolutionary Algorithms Yields Powerful Prompt Optimizers. ICLR 2024; preprint 2023.	Author preprint (https://arxiv.org/abs/2309.08532), proceedings (https://proceedings.iclr.cc/paper_files/paper/2024/hash/9156b0f6dfa9bbd18c79cc459ef5d61c-Abstract-Conference.html)	Prior prompt evolution; here the instruction is fixed.
2	Fernando et al. Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. ICML 2024.	PMLR (https://proceedings.mlr.press/v235/fernando24a.html)	Prior adaptation of mutation prompts; no claim of inventing adaptive mutation.
3	Lange, Tian, Tang. Large Language Models As Evolution Strategies. Preprint 2024.	arXiv (https://arxiv.org/abs/2402.18381)	LLMs as recombination operators; ours is conventional CPU tree search.
4	Liu et al. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. 2024.	Author preprint (https://arxiv.org/abs/2401.02051)	Prior search over executable heuristics; no heuristic-design novelty claimed.
5	Agrawal et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. Preprint 2025, v2 February 2026.	arXiv (https://arxiv.org/abs/2507.19457v2)	Reflective feedback and complementary candidate lessons are close conceptual precedents.
6	Ma et al. Eureka: Human-Level Reward Design via Coding Large Language Models. ICLR 2024; preprint 2023.	Author preprint (https://arxiv.org/abs/2310.12931)	Evolutionary reward-code generation; distinct task and representation.
7	Hu, Lu, Clune. Automated Design of Agentic Systems. ICLR 2025; preprint 2024.	Author preprint (https://arxiv.org/abs/2408.08435)	Search over agents; our inference features and actions are deliberately bounded.
8	Akiba et al. Evolutionary optimization of model merging recipes. Nature Machine Intelligence 7, 195 –204, 2025.	Publisher (https://www.nature.com/articles/s42256-024-00975-8)	Evolutionary model composition; ours changes presentation queries with frozen weights.
9	Wu et al. Evolutionary Computation in the Era of Large Language Model: Survey and Roadmap. Preprint 2024; IEEE TEC 2025 (course brief citation).	Author preprint (https://arxiv.org/abs/2401.10034)	Frames both directions of the EC–LLM intersection.
10	Yang et al. Large Language Models as Optimizers. ICLR 2024; preprint 2023, revised April 2024.	Author preprint (https://arxiv.org/abs/2309.03409)	Score-history-driven optimization; ours uses structured search-only error statistics.
11	Khattab et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. ICLR 2024; preprint 2023.	Author preprint (https://arxiv.org/abs/2310.03714)	Compound program optimization; no new claim for program-level search.
12	Opsahl-Ong et al. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. EMNLP 2024.	ACL proceedings (https://aclanthology.org/2024.emnlp-main.525/)	MIPRO credit assignment and surrogate optimization; not reproduced.
13	Yuksekgonul et al. TextGrad: Automatic Differentiation via Text. Preprint 2024.	Author preprint (https://arxiv.org/abs/2406.07496)	Rich textual optimization feedback; our operator uses no extra inference feedback.
14	Sclar, Choi, Tsvetkov, Suhr. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design. ICLR 2024; preprint revised July 2024.	Author preprint (https://arxiv.org/abs/2310.11324)	Format sensitivity/FormatSpread; formatting effects already established.
15	Wang et al. Look at the Text: Instruction-Tuned Language Models are More Robust Multiple Choice Selectors than You Think. COLM 2024.	Author preprint (https://arxiv.org/abs/2404.08382)	Generated-text versus first-token evaluation motivates response-contract limits.
16	Ong et al. RouteLLM: Learning to Route LLMs with Preference Data. Preprint 2024, revised 2025.	arXiv

(https://arxiv.org/abs/2406.18665)|Cost-quality routing prior; uses preferences and model routing, not format trees.|
17	Hu et al. RouterBench: A Benchmark for Multi-LLM Routing System. Preprint 2024.	arXiv (https://arxiv.org/abs/2403.12031)	Cached inference outcomes for routing evaluation; supports replay methodology but not live latency claims.
18	Wang et al. Mixture-of-Agents Enhances Large Language Model Capabilities. Preprint 2024.	arXiv (https://arxiv.org/abs/2406.04692)	Layered multi-model aggregation; our voting requires no aggregation-model call.
19	Li, Lin, Xia, Jin. Rethinking Mixture-of-Agents: Is Mixing Different Large Language Models Beneficial? Preprint 2025.	arXiv (https://arxiv.org/abs/2502.00674)	Self-MoA studies diversity-quality trade-off; useful warning against assuming complementarity helps.
20	Wang, Prasad, Stengel-Eskin, Bansal. Soft Self-Consistency Improves Language Model Agents. ACL 2024.	ACL proceedings (https://aclanthology.org/2024.acl-short.28/)	Soft scoring for varied valid outputs; ours has four fixed canonical identities. ACL web title says “Models Agents”; PDF/preprint use “Model Agents.”
21	Snell, Lee, Xu, Kumar. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. Preprint 2024.	arXiv (https://arxiv.org/abs/2408.03314)	Difficulty-dependent allocation; no claim of inventing adaptive test-time compute.
22	Kim, Yang, Min, Jung. Reliability-Aware Adaptive Self-Consistency for Efficient Sampling in LLM Reasoning. 2026, v2 April, ACL 2026 per author record.	arXiv (https://arxiv.org/abs/2601.02970)	Reliability-based stopping; omitted confidence features limit comparability.

Older foundations and assets, excluded from recent-paper count

- Aggarwal, Madaan, Yang, Mausam. Let's Sample Step by Step: Adaptive-Consistency for Efficient Reasoning and Coding with LLMs. EMNLP 2023. Primary paper (https://arxiv.org/abs/2305.11860). Prior adaptive agreement sampling; the fixed rule here is a transparent simple control, not a full reproduction.
- Chen, Zaharia, Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. 2023. Primary preprint (https://arxiv.org/abs/2305.05176). Learned cost-quality cascades precede this work.
- Zhou, Wu, Tang. Ensembling neural networks: Many could be better than all. Artificial Intelligence 137, 239–263, 2002. Publisher (https://www.sciencedirect.com/science/article/pii/S000437020200190X), author code (https://www.lamda.nju.edu.cn/code_GASEN.ashx). Genetic selective ensembles are long established.
- Deb, Pratap, Agarwal, Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE TEC 6, 182–197, 2002. Author publication record (https://www.egr.msu.edu/~kdeb/reports.shtml), DOI 10.1109/4235.996017. We use non-dominated sorting and crowding but a simple front-construction implementation and rank/hash tournament; call it NSGA-II-style, not an exact implementation.
- Pezeshkpour, Hruschka. Large Language Models Sensitivity to the Order of Options in Multiple-Choice Questions. 2023. Primary preprint (https://arxiv.org/abs/2308.11483). Option rotation sensitivity is prior work.
- Mihaylov, Clark, Khot, Sabharwal. Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering. EMNLP 2018. Paper (https://arxiv.org/abs/1809.02789), official code/data (https://github.com/allenai/OpenBookQA). This project uses questions and choices without supplying the annotated supporting fact, so it evaluates closed-book answering on OpenBookQA.
- Qwen2.5-3B-Instruct official model card (https://huggingface.co/Qwen/Qwen2.5-3B-Instruct). Model/tokenizer revision aa8e72537993ba99e69dfaafa59ed015b17504d1; fixed instruction and weights.

TARE is excluded because the previous project documented conflicting withdrawal/status records. No empirical TARE claim supports this project.